

20 Dimensionality Reduction

A common form of unsupervised learning is **dimensionality reduction**, in which we learn a mapping from the high-dimensional visible space, $\mathbf{x} \in \mathbb{R}^D$, to a low-dimensional latent space, $\mathbf{z} \in \mathbb{R}^L$. This mapping can either be a parametric model $\mathbf{z} = f(\mathbf{x}; \boldsymbol{\theta})$ which can be applied to any input, or it can be a nonparametric mapping where we compute an **embedding** $\mathbf{z}_n$ for each input $\mathbf{x}_n$ in the data set, but not for any other points. This latter approach is mostly used for data visualization, whereas the former approach can also be used as a preprocessing step for other kinds of learning algorithms. For example, we might first reduce the dimensionality by learning a mapping from $\mathbf{x}$ to $\mathbf{z}$, and then learn a simple linear classifier on this embedding, by mapping $\mathbf{z}$ to y .

20.1 Principal components analysis (PCA)

The simplest and most widely used form of dimensionality reduction is **principal components analysis** or **PCA**. The basic idea is to find a linear and orthogonal projection of the high dimensional data $\mathbf{x} \in \mathbb{R}^D$ to a low dimensional subspace $\mathbf{z} \in \mathbb{R}^L$, such that the low dimensional representation is a “good approximation” to the original data, in the following sense: if we project or **encode** $\mathbf{x}$ to get $\mathbf{z} = \mathbf{W}^\top \mathbf{x}$, and then unproject or **decode** $\mathbf{z}$ to get $\hat{\mathbf{x}} = \mathbf{W}\mathbf{z}$, then we want $\hat{\mathbf{x}}$ to be close to $\mathbf{x}$ in ℓ_2 distance. In particular, we can define the following **reconstruction error** or **distortion**:

$$\mathcal{L}(\mathbf{W}) \triangleq \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \text{decode}(\text{encode}(\mathbf{x}_n; \mathbf{W}); \mathbf{W})\|_2^2 \quad (20.1)$$

where the encode and decoding stages are both linear maps, as we explain below.

In Section 20.1.2, we show that we can minimize this objective by setting $\hat{\mathbf{W}} = \mathbf{U}_L$, where $\mathbf{U}_L$ contains the L eigenvectors with largest eigenvalues of the empirical covariance matrix

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top = \frac{1}{N} \mathbf{X}_c^\top \mathbf{X}_c \quad (20.2)$$

where $\mathbf{X}_c$ is a centered version of the $N \times D$ design matrix. In Section 20.2.2, we show that this is equivalent to maximizing the likelihood of a latent linear Gaussian model known as probabilistic PCA.

20.1.1 Examples

Before giving the details, we start by showing some examples.

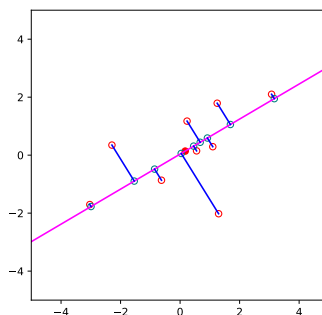


Figure 20.1: An illustration of PCA where we project from 2d to 1d. Red circles are the original data points, blue circles are the reconstructions. The red dot is the data mean. Generated by [pcaDemo2d.ipynb](#).

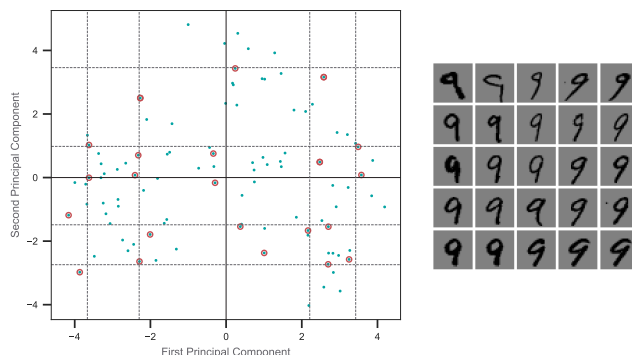


Figure 20.2: An illustration of PCA applied to MNIST digits from class 9. Grid points are at the 5, 25, 50, 75, 95 % quantiles of the data distribution along each dimension. The circled points are the closest projected images to the vertices of the grid. Adapted from Figure 14.23 of [HTF09]. Generated by [pca_digits.ipynb](#).

Figure 20.1 shows a very simple example, where we project 2d data to a 1d line. This direction captures most of the variation in the data.

In Figure 20.2, we show what happens when we project some MNIST images of the digit 9 down to 2d. Although the inputs are high dimensional (specifically $28 \times 28 = 784$ dimensional), the number of “effective degrees of freedom” is much less, since the pixels are correlated, and many digits look similar. Therefore we can represent each image as a point in a low dimensional linear space.

In general, it can be hard to interpret the latent dimensions to which the data is projected. However, by looking at several projected points along a given direction, and the examples from which they are derived, we see that the first principal component (horizontal direction) seems to capture the orientation of the digit, and the second component (vertical direction) seems to capture line thickness.

In Figure 20.3, we show PCA applied to another image dataset, known as the Olivetti face dataset,



Figure 20.3: a) Some randomly chosen 64×64 pixel images from the Olivetti face database. (b) The mean and the first three PCA components represented as images. Generated by [pcaImageDemo.ipynb](#).

which is a set of 64×64 grayscale images. We project these to a 3d subspace. The resulting basis vectors (columns of the projection matrix $\mathbf{W}$) are shown as images in Figure 20.3b; these are known as **eigenfaces** [Tur13], for reasons that will be explained in Section 20.1.2. We see that the main modes of variation in the data are related to overall lighting, and then differences in the eyebrow region of the face. If we use enough dimensions (but fewer than the 4096 we started with), we can use the representation $\mathbf{z} = \mathbf{W}^T \mathbf{x}$ as input to a nearest-neighbor classifier to perform face recognition; this is faster and more reliable than working in pixel space [MWP98].

20.1.2 Derivation of the algorithm

Suppose we have an (unlabeled) dataset $\mathcal{D} = \{\mathbf{x}_n : n = 1 : N\}$, where $\mathbf{x}_n \in \mathbb{R}^D$. We can represent this as an $N \times D$ data matrix $\mathbf{X}$. We will assume $\bar{\mathbf{x}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n = \mathbf{0}$, which we can ensure by centering the data.

We would like to approximate each $\mathbf{x}_n$ by a low dimensional representation, $\mathbf{z}_n \in \mathbb{R}^L$. We assume that each $\mathbf{x}_n$ can be “explained” in terms of a weighted combination of basis functions $\mathbf{w}_1, \dots, \mathbf{w}_L$, where each $\mathbf{w}_k \in \mathbb{R}^D$, and where the weights are given by $\mathbf{z}_n \in \mathbb{R}^L$, i.e., we assume $\mathbf{x}_n \approx \sum_{k=1}^L z_{nk} \mathbf{w}_k$. The vector $\mathbf{z}_n$ is the low dimensional representation of $\mathbf{x}_n$, and is known as the **latent vector**, since it consists of latent or “hidden” values that are not observed in the data. The collection of these latent variables are called the **latent factors**.

We can measure the error produced by this approximation as follows:

$$\mathcal{L}(\mathbf{W}, \mathbf{Z}) = \frac{1}{N} \|\mathbf{X} - \mathbf{Z}\mathbf{W}^T\|_F^2 = \frac{1}{N} \|\mathbf{X}^T - \mathbf{W}\mathbf{Z}^T\|_F^2 = \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \mathbf{W}\mathbf{z}_n\|^2 \quad (20.3)$$

where the rows of $\mathbf{Z}$ contain the low dimension versions of the rows of $\mathbf{X}$. This is known as the (average) **reconstruction error**, since we are approximating each $\mathbf{x}_n$ by $\hat{\mathbf{x}}_n = \mathbf{W}\mathbf{z}_n$.

We want to minimize this subject to the constraint that $\mathbf{W}$ is an orthogonal matrix. Below we show that the optimal solution is obtained by setting $\hat{\mathbf{W}} = \mathbf{U}_L$, where $\mathbf{U}_L$ contains the L eigenvectors with largest eigenvalues of the empirical covariance matrix.

20.1.2.1 Base case

Let us start by estimating the best 1d solution, $\mathbf{w}_1 \in \mathbb{R}^D$. We will find the remaining basis vectors $\mathbf{w}_2, \mathbf{w}_3$, etc. later.

Let the coefficients for each of the data points associated with the first basis vector be denoted by $\tilde{\mathbf{z}}_1 = [z_{11}, \dots, z_{N1}] \in \mathbb{R}^N$. The reconstruction error is given by

$$\mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1) = \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - z_{n1} \mathbf{w}_1\|^2 = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - z_{n1} \mathbf{w}_1)^\top (\mathbf{x}_n - z_{n1} \mathbf{w}_1) \quad (20.4)$$

$$= \frac{1}{N} \sum_{n=1}^N [\mathbf{x}_n^\top \mathbf{x}_n - 2z_{n1} \mathbf{w}_1^\top \mathbf{x}_n + z_{n1}^2 \mathbf{w}_1^\top \mathbf{w}_1] \quad (20.5)$$

$$= \frac{1}{N} \sum_{n=1}^N [\mathbf{x}_n^\top \mathbf{x}_n - 2z_{n1} \mathbf{w}_1^\top \mathbf{x}_n + z_{n1}^2] \quad (20.6)$$

since $\mathbf{w}_1^\top \mathbf{w}_1 = 1$ (by the orthonormality assumption). Taking derivatives wrt z_{n1} and equating to zero gives

$$\frac{\partial}{\partial z_{n1}} \mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1) = \frac{1}{N} [-2\mathbf{w}_1^\top \mathbf{x}_n + 2z_{n1}] = 0 \Rightarrow z_{n1} = \mathbf{w}_1^\top \mathbf{x}_n \quad (20.7)$$

So the optimal embedding is obtained by orthogonally projecting the data onto $\mathbf{w}_1$ (see Figure 20.1). Plugging this back in gives the loss for the weights:

$$\mathcal{L}(\mathbf{w}_1) = \mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1^*(\mathbf{w}_1)) = \frac{1}{N} \sum_{n=1}^N [\mathbf{x}_n^\top \mathbf{x}_n - z_{n1}^2] = \text{const} - \frac{1}{N} \sum_{n=1}^N z_{n1}^2 \quad (20.8)$$

To solve for $\mathbf{w}_1$, note that

$$\mathcal{L}(\mathbf{w}_1) = -\frac{1}{N} \sum_{n=1}^N z_{n1}^2 = -\frac{1}{N} \sum_{n=1}^N \mathbf{w}_1^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w}_1 = -\mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 \quad (20.9)$$

where Σ is the empirical covariance matrix (since we assumed the data is centered). We can trivially optimize this by letting $\|\mathbf{w}_1\| \rightarrow \infty$, so we impose the constraint $\|\mathbf{w}_1\| = 1$ and instead optimize

$$\tilde{\mathcal{L}}(\mathbf{w}_1) = \mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 - \lambda_1 (\mathbf{w}_1^\top \mathbf{w}_1 - 1) \quad (20.10)$$

where λ_1 is a Lagrange multiplier (see Section 8.5.1). Taking derivatives and equating to zero we have

$$\frac{\partial}{\partial \mathbf{w}_1} \tilde{\mathcal{L}}(\mathbf{w}_1) = 2\hat{\Sigma} \mathbf{w}_1 - 2\lambda_1 \mathbf{w}_1 = 0 \quad (20.11)$$

$$\hat{\Sigma} \mathbf{w}_1 = \lambda_1 \mathbf{w}_1 \quad (20.12)$$

Hence the optimal direction onto which we should project the data is an eigenvector of the covariance matrix. Left multiplying by $\mathbf{w}_1^\top$ (and using $\mathbf{w}_1^\top \mathbf{w}_1 = 1$) we find

$$\mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 = \lambda_1 \quad (20.13)$$

Since we want to maximize this quantity (minimize the loss), we pick the eigenvector which corresponds to the largest eigenvalue.

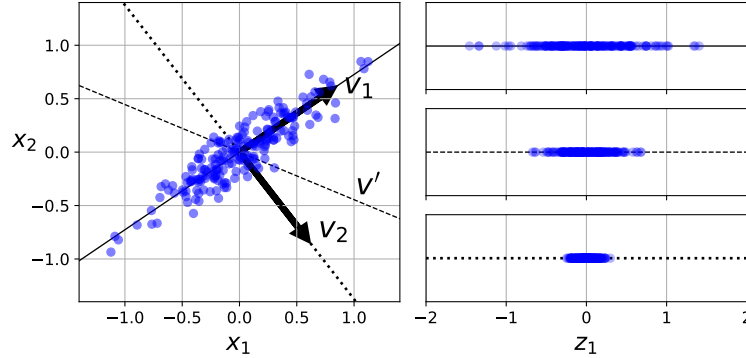


Figure 20.4: Illustration of the variance of the points projected onto different 1d vectors. v_1 is the first principal component, which maximizes the variance of the projection. v_2 is the second principal component which is direction orthogonal to v_1 . Finally v' is some other vector in between v_1 and v_2 . Adapted from Figure 8.7 of [Gér19]. Generated by `pca_projected_variance.ipynb`

20.1.2.2 Optimal weight vector maximizes the variance of the projected data

Before continuing, we make an interesting observation. Since the data has been centered, we have

$$\mathbb{E}[z_{n1}] = \mathbb{E}[\mathbf{x}_n^\top \mathbf{w}_1] = \mathbb{E}[\mathbf{x}_n]^\top \mathbf{w}_1 = 0 \quad (20.14)$$

Hence variance of the projected data is given by

$$\mathbb{V}[\tilde{\mathbf{z}}_1] = \mathbb{E}[\tilde{\mathbf{z}}_1^2] - (\mathbb{E}[\tilde{\mathbf{z}}_1])^2 = \frac{1}{N} \sum_{n=1}^N z_{n1}^2 - 0 = -\mathcal{L}(\mathbf{w}_1) + \text{const} \quad (20.15)$$

From this, we see that *minimizing* the reconstruction error is equivalent to *maximizing* the variance of the projected data:

$$\arg \min_{\mathbf{w}_1} \mathcal{L}(\mathbf{w}_1) = \arg \max_{\mathbf{w}_1} \mathbb{V}[\tilde{\mathbf{z}}_1(\mathbf{w}_1)] \quad (20.16)$$

This is why it is often said that PCA finds the directions of maximal variance. (See Figure 20.4 for an illustration.) However, the minimum error formulation is easier to understand and is more general.

20.1.2.3 Induction step

Now let us find another direction $\mathbf{w}_2$ to further minimize the reconstruction error, subject to $\mathbf{w}_1^\top \mathbf{w}_2 = 0$ and $\mathbf{w}_2^\top \mathbf{w}_2 = 1$. The error is

$$\mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1, \mathbf{w}_2, \tilde{\mathbf{z}}_2) = \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - z_{n1} \mathbf{w}_1 - z_{n2} \mathbf{w}_2\|^2 \quad (20.17)$$

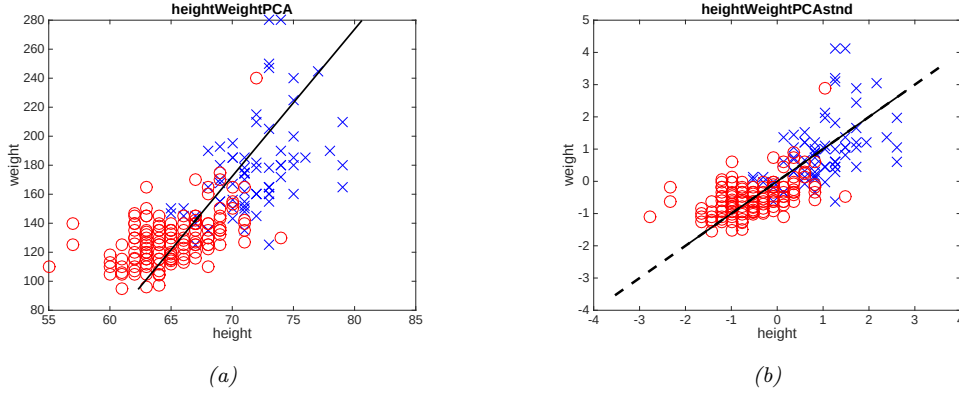


Figure 20.5: Effect of standardization on PCA applied to the height/weight dataset. (Red=female, blue=male.) Left: PCA of raw data. Right: PCA of standardized data. Generated by [pcaStandardization.ipynb](#).

Optimizing wrt $\mathbf{w}_1$ and $\mathbf{z}_1$ gives the same solution as before. Exercise 20.3 asks you to show that $\frac{\partial \mathcal{L}}{\partial \mathbf{z}_2} = 0$ yields $z_{n2} = \mathbf{w}_2^\top \mathbf{x}_n$. Substituting in yields

$$\mathcal{L}(\mathbf{w}_2) = \frac{1}{N} \sum_{n=1}^N [\mathbf{x}_n^\top \mathbf{x}_n - \mathbf{w}_1^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w}_1 - \mathbf{w}_2^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w}_2] = \text{const} - \mathbf{w}_2^\top \hat{\Sigma} \mathbf{w}_2 \quad (20.18)$$

Dropping the constant term, plugging in the optimal $\mathbf{w}_1$ and adding the constraints yields

$$\tilde{\mathcal{L}}(\mathbf{w}_2) = -\mathbf{w}_2^\top \hat{\Sigma} \mathbf{w}_2 + \lambda_2 (\mathbf{w}_2^\top \mathbf{w}_2 - 1) + \lambda_{12} (\mathbf{w}_2^\top \mathbf{w}_1 - 0) \quad (20.19)$$

Exercise 20.3 asks you to show that the solution is given by the eigenvector with the second largest eigenvalue:

$$\hat{\Sigma} \mathbf{w}_2 = \lambda_2 \mathbf{w}_2 \quad (20.20)$$

The proof continues in this way to show that $\hat{\mathbf{W}} = \mathbf{U}_L$.

20.1.3 Computational issues

In this section, we discuss various practical issues related to using PCA.

20.1.3.1 Covariance matrix vs correlation matrix

We have been working with the eigendecomposition of the covariance matrix. However, it is better to use the correlation matrix instead. The reason is that otherwise PCA can be “misled” by directions in which the variance is high merely because of the measurement scale. Figure 20.5 shows an example of this. On the left, we see that the vertical axis uses a larger range than the horizontal axis. This results in a first principal component that looks somewhat “unnatural”. On the right, we show the results of PCA after standardizing the data (which is equivalent to using the correlation matrix instead of the covariance matrix); the results look much better.

20.1.3.2 Dealing with high-dimensional data

We have presented PCA as the problem of finding the eigenvectors of the $D \times D$ covariance matrix $\mathbf{X}^T \mathbf{X}$. If $D > N$, it is faster to work with the $N \times N$ Gram matrix $\mathbf{X} \mathbf{X}^T$. We now show how to do this.

First, let $\mathbf{U}$ be an orthonormal matrix containing the eigenvectors of $\mathbf{X} \mathbf{X}^T$ with corresponding eigenvalues in Λ . By definition we have $(\mathbf{X} \mathbf{X}^T) \mathbf{U} = \mathbf{U} \Lambda$. Pre-multiplying by $\mathbf{X}^T$ gives

$$(\mathbf{X}^T \mathbf{X})(\mathbf{X}^T \mathbf{U}) = (\mathbf{X}^T \mathbf{U}) \Lambda \quad (20.21)$$

from which we see that the eigenvectors of $\mathbf{X}^T \mathbf{X}$ are $\mathbf{V} = \mathbf{X}^T \mathbf{U}$, with eigenvalues given by Λ as before. However, these eigenvectors are not normalized, since $\|\mathbf{v}_j\|^2 = \mathbf{u}_j^T \mathbf{X} \mathbf{X}^T \mathbf{u}_j = \lambda_j \mathbf{u}_j^T \mathbf{u}_j = \lambda_j$. The normalized eigenvectors are given by

$$\mathbf{V} = \mathbf{X}^T \mathbf{U} \Lambda^{-\frac{1}{2}} \quad (20.22)$$

This provides an alternative way to compute the PCA basis. It also allows us to use the kernel trick, as we discuss in Section 20.4.6.

20.1.3.3 Computing PCA using SVD

In this section, we show the equivalence between PCA as computed using eigenvector methods (Section 20.1) and the truncated SVD.¹

Let $\mathbf{U}_\Sigma \Lambda_\Sigma \mathbf{U}_\Sigma^T$ be the top L eigendecomposition of the covariance matrix $\Sigma \propto \mathbf{X}^T \mathbf{X}$ (we assume $\mathbf{X}$ is centered). Recall from Section 20.1.2 that the optimal estimate of the projection weights $\mathbf{W}$ is given by the top L eigenvalues, so $\mathbf{W} = \mathbf{U}_\Sigma$.

Now let $\mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^T \approx \mathbf{X}$ be the L -truncated SVD approximation to the data matrix $\mathbf{X}$. From Equation (7.184), we know that the right singular vectors of $\mathbf{X}$ are the eigenvectors of $\mathbf{X}^T \mathbf{X}$, so $\mathbf{V}_X = \mathbf{U}_\Sigma = \mathbf{W}$. (In addition, the eigenvalues of the covariance matrix are related to the singular values of the data matrix via $\lambda_k = s_k^2/N$.)

Now suppose we are interested in the projected points (also called the principal components or PC scores), rather than the projection matrix. We have

$$\mathbf{Z} = \mathbf{X} \mathbf{W} = \mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^T \mathbf{V}_X = \mathbf{U}_X \mathbf{S}_X \quad (20.23)$$

Finally, if we want to approximately reconstruct the data, we have

$$\hat{\mathbf{X}} = \mathbf{Z} \mathbf{W}^T = \mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^T \quad (20.24)$$

This is precisely the same as a truncated SVD approximation (Section 7.5.5).

Thus we see that we can perform PCA either using an eigendecomposition of Σ or an SVD decomposition of $\mathbf{X}$. The latter is often preferable, for computational reasons. For very high dimensional problems, we can use a randomized SVD algorithm, see e.g., [HMT11; SKT14; DM16]. For example, the randomized solver used by sklearn takes $O(NL^2) + O(L^3)$ time for N examples and L principal components, whereas exact SVD takes $O(ND^2) + O(D^3)$ time.

1. A more detailed explanation can be found at <https://bit.ly/2I5660K>.

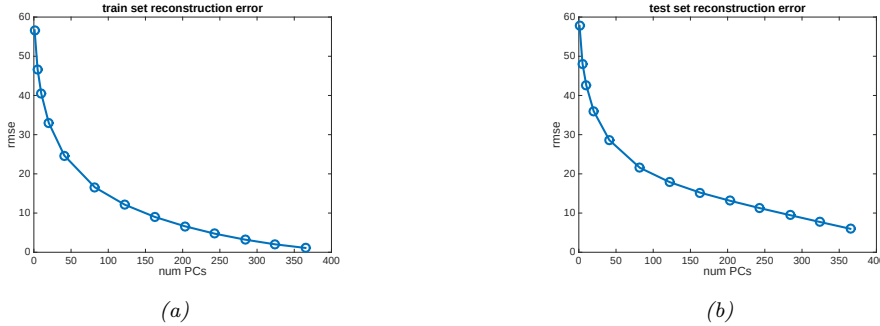


Figure 20.6: Reconstruction error on MNIST vs number of latent dimensions used by PCA. (a) Training set. (b) Test set. Generated by [pcaOverfitDemo.ipynb](#).

20.1.4 Choosing the number of latent dimensions

In this section, we discuss how to choose the number of latent dimensions L for PCA.

20.1.4.1 Reconstruction error

Let us define the reconstruction error on some dataset $\mathcal{D}$ incurred by the model when using L dimensions:

$$\mathcal{L}_L = \frac{1}{|\mathcal{D}|} \sum_{n \in \mathcal{D}} \|x_n - \hat{x}_n\|^2 \quad (20.25)$$

where the reconstruction is given by $\hat{x}_n = \mathbf{W}z_n + \boldsymbol{\mu}$, where $z_n = \mathbf{W}^\top(x_n - \boldsymbol{\mu})$ and $\boldsymbol{\mu}$ is the empirical mean, and $\mathbf{W}$ is estimated as above. Figure 20.6(a) plots $\mathcal{L}_L$ vs L on the MNIST training data. We see that it drops off quite quickly, indicating that we can capture most of the empirical correlation of the pixels with a small number of factors.

Of course, if we use $L = \text{rank}(\mathbf{X})$, we get zero reconstruction error on the training set. To avoid overfitting, it is natural to plot reconstruction error on the test set. This is shown in Figure 20.6(b). Here we see that the error continues to go down even as the model becomes more complex! Thus we do not get the usual U-shaped curve that we typically expect to see in supervised learning. The problem is that PCA is not a proper generative model of the data: If you give it more latent dimensions, it will be able to approximate the test data more accurately. (A similar problem arises if we plot reconstruction error on the test set using K-means clustering, as discussed in Section 21.3.7.) We discuss some solutions to this below.

20.1.4.2 Scree plots

A common alternative to plotting reconstruction error vs L is to use something called a **scree plot**, which is a plot of the eigenvalues λ_j vs j in order of decreasing magnitude. One can show

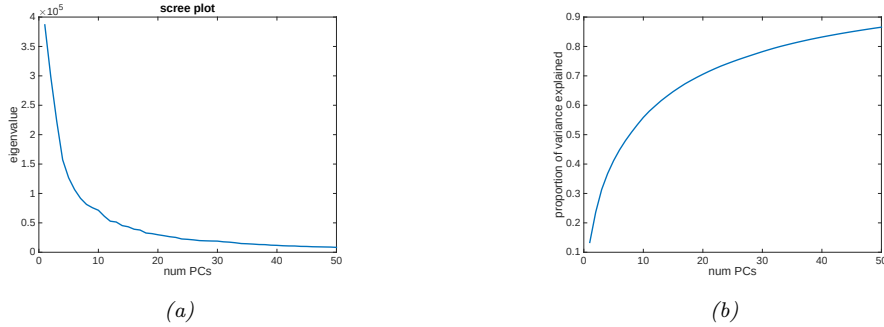


Figure 20.7: (a) Scree plot for training set, corresponding to Figure 20.6(a). (b) Fraction of variance explained. Generated by `pcaOverfitDemo.ipynb`.

(Exercise 20.4) that

$$\mathcal{L}_L = \sum_{j=L+1}^D \lambda_j \quad (20.26)$$

Thus as the number of dimensions increases, the eigenvalues get smaller, and so does the reconstruction error, as shown in Figure 20.7a.² A related quantity is the **fraction of variance explained**, defined as

$$F_L = \frac{\sum_{j=1}^L \lambda_j}{\sum_{j'=1}^{L_{\max}} \lambda_{j'}} \quad (20.27)$$

This captures the same information as the scree plot, but goes up with L (see Figure 20.7b).

20.1.4.3 Profile likelihood

Although there is no U-shape in the reconstruction error plot, there is sometimes a “knee” or “elbow” in the curve, where the error suddenly changes from relatively large errors to relatively small. The idea is that for $L < L^*$, where L^* is the “true” latent dimensionality (or number of clusters), the rate of decrease in the error function will be high, whereas for $L > L^*$, the gains will be smaller, since the model is already sufficiently complex to capture the true distribution.

One way to automate the detection of this change in the gradient of the curve is to compute the **profile likelihood**, as proposed in [ZG06]. The idea is this. Let λ_L be some measure of the error incurred by a model of size L , such that $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{L_{\max}}$. In PCA, these are the eigenvalues, but the method can also be applied to the reconstruction error from K-means clustering (see Section 21.3.7). Now consider partitioning these values into two groups, depending on whether $k < L$ or $k > L$, where L is some threshold which we will determine. To measure the quality of L ,

2. The reason for the term “scree plot” is that “the plot looks like the side of a mountain, and ‘scree’ refers to the debris fallen from a mountain and lying at its base”. (Quotation from Kenneth Janda, <https://bit.ly/2kqG1yW>.)